But that's not all that's wrong with that simple update equation. Because it involves the max function, i.e., we select the maximum predicted Q value for the next state, it leads our agent to overestimate Q values for actions, which can impact training especially early on. If the DQN takes action 1 and learns an erroneously high Q value for action 1, that means action 1 is going to get selected more often in subsequent epochs, further causing it to be overestimated, which again leads to training instability and poor performance.

To mitigate this problem and get more accurate estimates for Q values, we will implement double Q-learning, which solves the problem by disentangling action-value estimation from action selection, as you will see. A *double deep Q-network* (DDQN) involves a simple modification to normal Q-learning with a target network. As usual, we use the main Q-network to select actions using an epsilon-greedy policy. But when it comes time to compute Q_{new}, we will first find the argmax of Q (the main Q-network). Let's say $\text{argmax}(Q(s_{t+1})) = 2$, so action 2 is associated with the highest action value in the next state given the main Q function. We then use this to index into the target network, Q', to get the action value we will use in the update equation.

$$a = \text{argmax}(Q(s_{t+1}))$$
$$x = Q'(s_{t+1})[a]$$
$$Q_{new} = r_t + \gamma \times x$$

We're still using the Q value from the target network, Q', but we don't choose the highest Q value from Q'; we choose the Q value in Q' based on the action associated with the highest Q value in the main Q function. In code:

```
>>> state_batch, action_batch, reward_batch, state2_batch,
done_batch = get_minibatch(replay, batch_size)
>>> q_pred = GWagent(state_batch)
>>> astar = torch.argmax(q_pred,dim=1)
>>> qs =
Tnet(state2_batch).gather(dim=1,index=astar.unsqueeze(dim=1)).squee
ze()
>>> targets =
get_qtarget_ddqn(qs.detach(),reward_batch.detach(),gamma,done_batch
)
```